CAPSTONE PROJECT 2

Retail Demand Forecasting & Inventory Optimization using SQL and Machine Learning.

EXECUTIVE SUMMARY:

This project focuses on building a retail demand forecasting and inventory optimization system using SQL and Machine Learning. The objective was to predict daily store-level sales using historical retail data and use those predictions to support inventory planning decisions.

The Rossmann Store Sales dataset was used for business analysis in SQL, data preprocessing, feature engineering, and model development in Python. Multiple regression models were trained and evaluated, and Random Forest achieved the best performance with an R^2 score close to 0.90.

Based on the forecasted demand, an inventory reorder logic using safety stock and reorder point calculations was implemented to reduce stock-outs and improve inventory efficiency.

1. PROBLEM STATEMENT:

Retail businesses often face challenges in predicting product demand accurately. Inaccurate forecasting can lead to stock-outs, resulting in lost sales and customer dissatisfaction, or overstocking, which increases storage costs and capital blockage.

The objective of this project is to develop an end-to-end system that forecasts future sales demand using historical retail data and recommends optimal inventory reorder levels. The solution combines SQL-based business analysis with machine learning models to improve demand visibility and support better inventory planning decision.

2. DATASET OVERVIEW:

The project uses the Rossmann Store Sales dataset, a publicly available retail dataset from Kaggle. The dataset contains daily sales information for 1,115 retail stores along with store metadata and promotional details.

After data cleaning and removing non-operational days, the final dataset contained approximately 836,533 records covering the period from January 2013 to July 2015. The dataset includes features such as Store, Date, Sales, Customers, Promo, StateHoliday, SchoolHoliday, StoreType, Assortment, and CompetitionDistance.

This dataset provides sufficient historical depth to capture seasonality, promotional impact, and store-level demand patterns required for time-series forecasting and inventory planning.

3. DATA CLEANING & PREPROCESSING:

The dataset was cleaned and preprocessed using Python before performing detailed analysis and modeling. The following steps were carried out to ensure data quality and consistency:

- Removed records where stores were closed (Open = 0)
- Removed entries with zero sales to avoid distortion in demand modeling
- Converted the Date column to datetime format
- Handled missing values in competition and promotion-related fields
- Filled CompetitionDistance using median imputation
- Replaced missing promotion-related time fields with zero
- Replaced missing PromoInterval values with "None"

After cleaning and removing non-operational days, the final dataset contained approximately 836,533 records, making it suitable for forecasting analysis.

4. FEATURE ENGINEERING:

To improve forecasting performance, additional time-based and demand-driven features were created from the existing dataset. These features help the model capture seasonality, weekly patterns, and short-term demand trends.

The following time-based features were extracted from the Date column:

- Year
- Month

- Week number
- Quarter
- DayOfWeek
- IsWeekend indicator

In addition to seasonal features, lag and rolling statistics were created to capture historical demand behaviour:

- Lag_7 (sales from 7 days ago)
- Rolling_Mean_7 (7-day moving average of sales)

The dataset was sorted by Store and Date before generating lag features to ensure correct time-series calculations. Rows with missing values created due to lag operations were removed.

These engineered features significantly improved the model's ability to capture short-term demand momentum and seasonal variations.

5. SQL-BASED BUSINESS ANALYSIS:

After cleaning and preprocessing the dataset in Python, SQL was used to perform structured business analysis on the processed retail data. The objective was to validate the cleaned dataset and extract business-level insights related to sales patterns, store performance, seasonality, and promotional impact.

Key analyses performed using SQL include:

- Total number of transactions and overall revenue
- Date range validation and store count verification
- Store-wise revenue contribution and ranking
- Monthly and yearly sales trend analysis
- Promotion impact on average sales
- Weekend vs weekday sales comparison
- Store type performance analysis
- Running total and month-over-month growth using window functions

Promotion Impact Analysis:

Objective: To analyze the impact of promotional activity on sales performance.

```
SELECT
    Promo,
    COUNT(*) AS transactions,
    SUM(Sales) AS total_sales,
    ROUND(AVG(Sales), 2) AS avg_sales
FROM retail_data
GROUP BY Promo;
```

Interpretation:

Promotional days generated significantly higher average sales compared to non-promotional days, indicating strong revenue uplift driven by promotions.

Store Ranking Using Window Function:

Objective: To rank stores based on total revenue contribution.

```
SELECT *
FROM (
    SELECT
        Store,
        SUM(Sales) AS total_sales,
        RANK() OVER (ORDER BY SUM(Sales) DESC) AS store_rank
    FROM retail_data
    GROUP BY Store
) t
WHERE store_rank <= 10;
```

Interpretation:

Revenue distribution is concentrated among top-performing stores, highlighting performance variation across different store locations.

Running Total of Monthly Sales

Objective: To analyze cumulative revenue growth over time.

```
WITH monthly_sales AS (
    SELECT
        YEAR(Date) AS Year,
        MONTH(Date) AS Month,
        SUM(Sales) AS total_sales
    FROM retail_data
    GROUP BY YEAR(Date), MONTH(Date)
)
SELECT
    Year,
    Month,
    total_sales,
    SUM(total_sales) OVER (
        ORDER BY Year, Month
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM monthly_sales;
```

Interpretation:

The running total shows a steady cumulative increase in revenue over time, reflecting consistent business growth with seasonal fluctuations.

The SQL analysis confirmed strong seasonal patterns, especially during year-end months, and highlighted the significant impact of promotions on revenue. These insights supported the need for a forecasting model that captures seasonality and demand fluctuations.

6. EXPLORATORY DATA ANALYSIS (EDA):

Exploratory Data Analysis was performed to understand sales distribution, seasonality patterns, promotional impact, and store-level performance before building predictive models.

Key observations from the analysis include:

- Sales distribution is right-skewed, indicating the presence of high-value sales days.

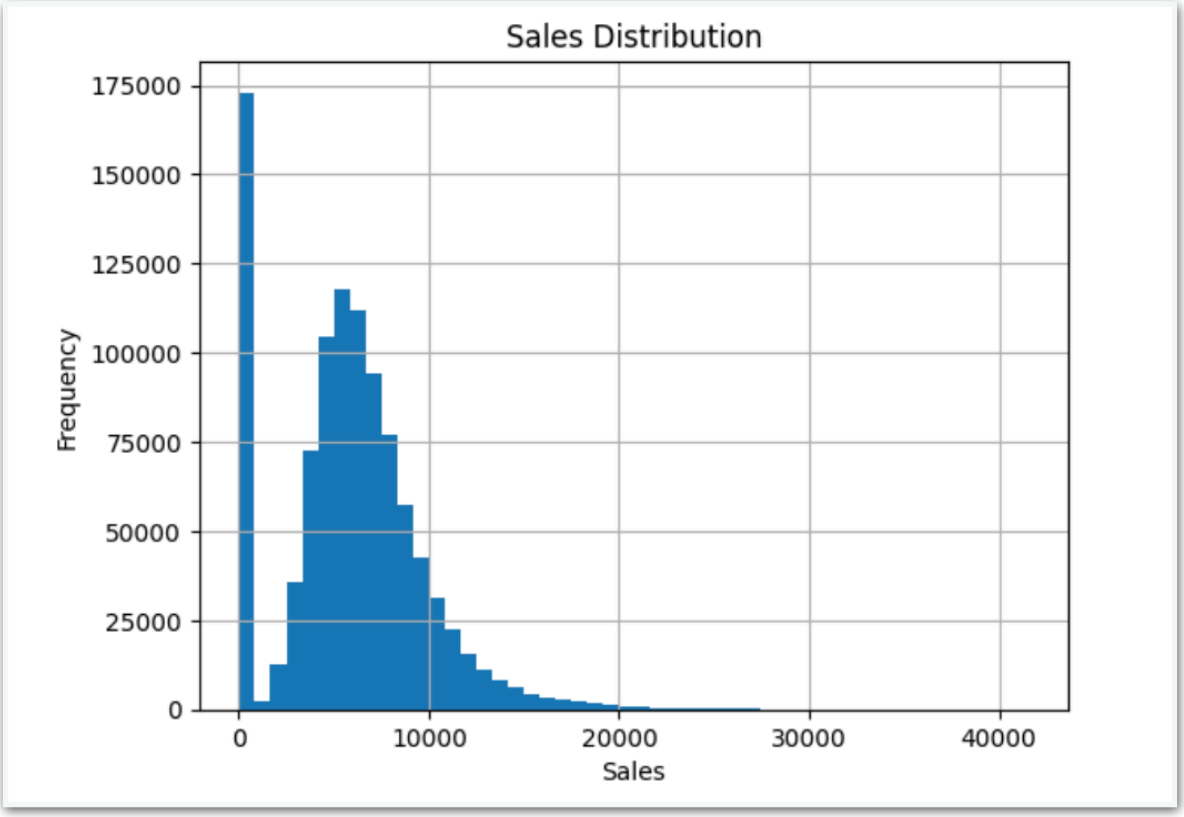

Figure 1: Distribution of Daily Sales (Right-Skewed Distribution

- Monthly sales trends show clear seasonal variation, with strong spikes during year-end months.

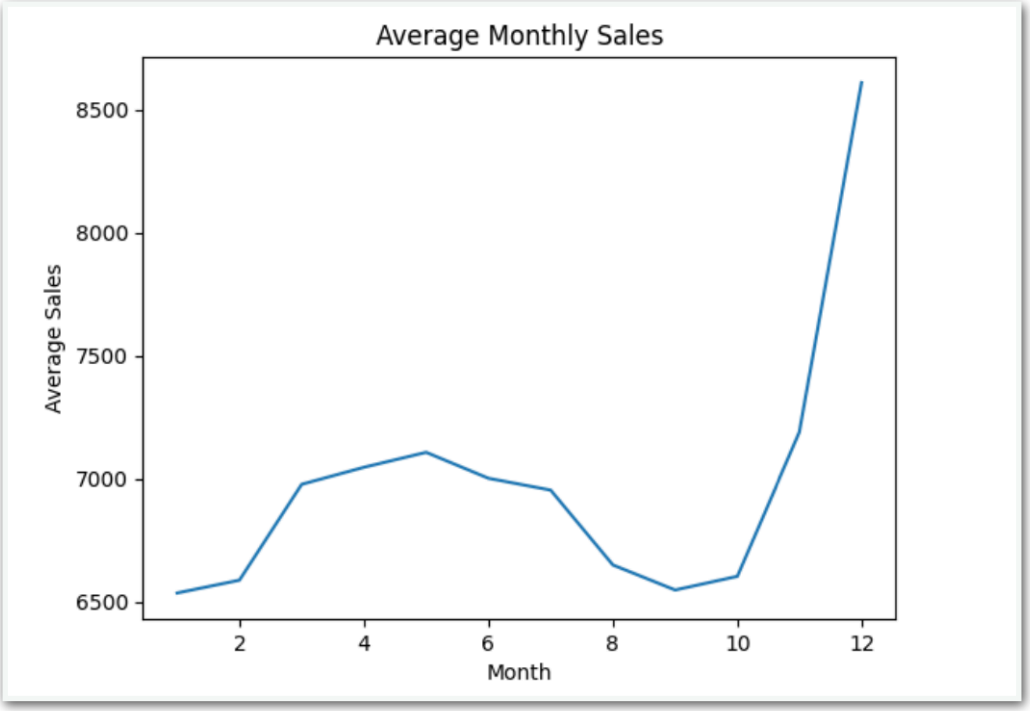

Figure 2: Average Monthly Sales Showing Seasonal Variation

- Promotional periods significantly increase average sales compared to non-promotional days.

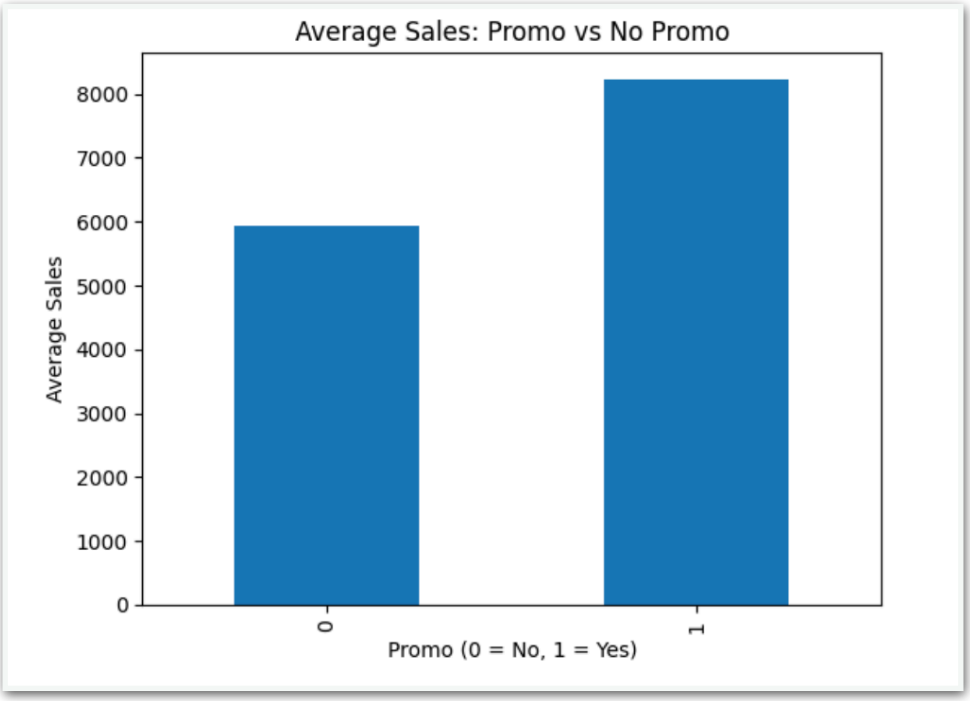

Figure 3: Impact of Promotional Activity on Average Sales

- Weekly patterns indicate cyclical demand behavior across days of the week.

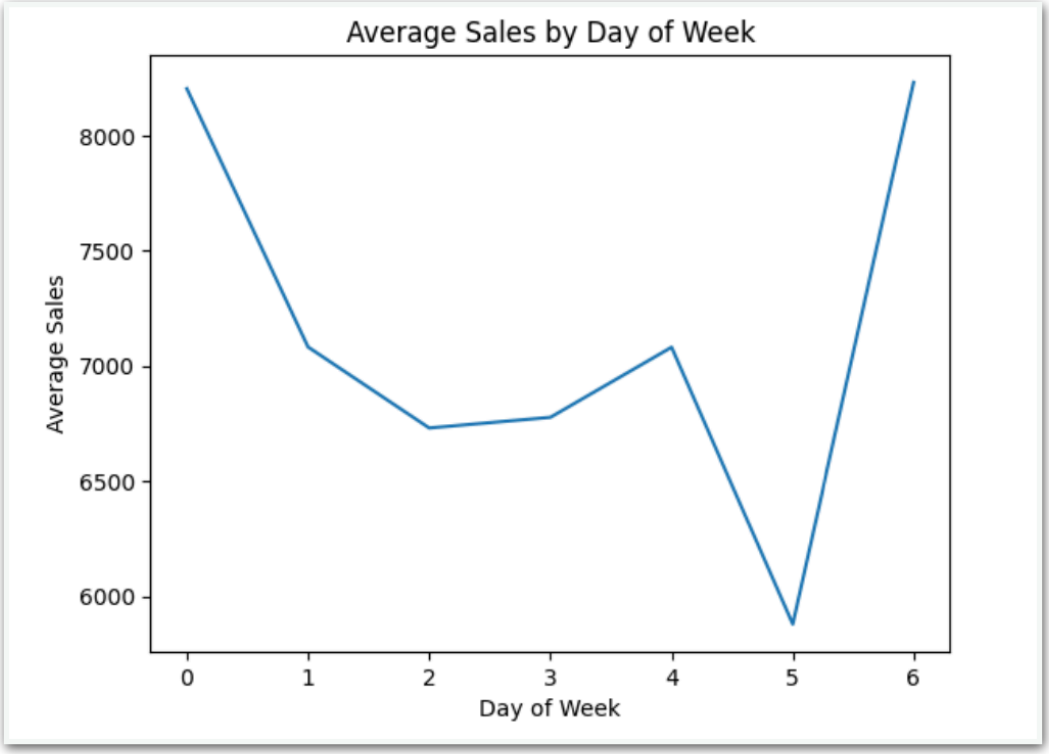

Figure 4: Average Sales by Day of Week

These insights confirmed the presence of seasonality and promotional influence in the dataset, supporting the need for time-based features and non-linear regression models.

7. MODEL DEVELOPMENT:

To forecast daily retail sales, multiple regression models were developed and compared. The dataset was prepared by encoding categorical variables and separating features (X) and target variable (y).

The dataset was split into training and testing sets using an 80:20 ratio to evaluate model performance on unseen data.

The following models were implemented:

- Linear Regression (baseline model)
- Decision Tree Regressor
- Random Forest Regressor
- XGBoost Regressor

Each model was evaluated using the following metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Linear Regression was used as a baseline model, while tree-based ensemble models were implemented to capture non-linear relationships and complex demand patterns.

8. MODEL EVALUATION & SELECTION:

The performance of all models was compared using MAE, RMSE, and R^2 metrics. Among the evaluated models, Random Forest Regressor achieved the lowest error values and the highest R^2 score (approximately 0.90), indicating strong predictive performance.

	Model	MAE	RMSE	R2 Score
0	Linear Regression	1219.694757	1668.355268	0.712447
1	Decision Tree	898.748247	1341.041643	0.814209
2	Random Forest	678.645438	995.380029	0.897643
3	XGBoost	698.952271	1003.543223	0.895957

Table 1: Model Performance Comparison

Linear Regression showed moderate performance but was limited in capturing non-linear patterns in the data. Decision Tree improved performance by modeling non-linear relationships, but it was prone to overfitting. XGBoost also performed well; however, Random Forest provided slightly better overall error reduction and model stability.

Based on the evaluation metrics and generalization performance, Random Forest Regressor was selected as the final forecasting model.

9. MODEL TUNING:

Hyperparameters for the Random Forest model were manually adjusted to improve predictive performance. Different combinations of n_estimators, max_depth, and min_samples_split were tested to balance model accuracy and generalization.

The final selected configuration (n_estimators = 50, max_depth = 18, min_samples_split = 10) achieved strong performance with an R^2 score close to 0.90.

Further aggressive tuning was not pursued, as the model already demonstrated strong predictive capability and additional complexity could increase the risk of overfitting.

10. FEATURE IMPORTANCE ANALYSIS:

Feature importance was analyzed using the final selected Random Forest model to understand which variables most influence sales predictions.

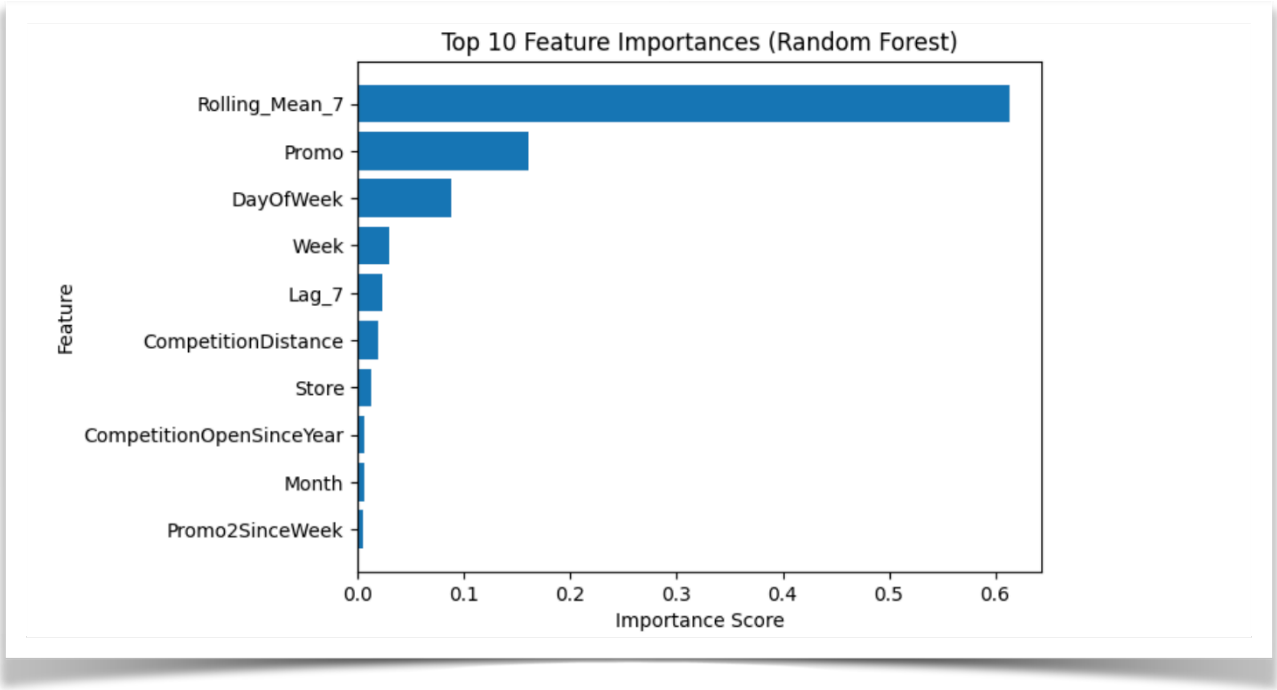

Figure 5: Top 10 Feature Importance – Random Forest Model

The results show that the 7-day rolling mean of sales (Rolling_Mean_7) is the most important feature, indicating strong short-term demand momentum. Promotional activity (Promo) also has a significant impact on sales performance.

Time-based features such as DayOfWeek and Week contribute to capturing weekly seasonality patterns. The presence of lag-based features (Lag_7) further confirms that recent historical sales strongly influence future demand.

These results validate the importance of temporal and promotional factors in retail demand forecasting.

11. INVENTORY OPTIMIZATION:

Using the forecasted demand, an inventory reorder strategy was designed to minimize stock-outs and excess holding costs. The objective was to translate predicted sales into actionable inventory planning decisions.

Inventory decisions were based on the following logic:

Reorder Point = (Average Daily Demand × Lead Time) + Safety Stock

Safety Stock = Z × Demand Standard Deviation × √Lead Time

Store	Average_Daily_Demand	Demand_StdDev	Safety_Stock	Reorder_Point
1	4753.600775	1011.603204	4416.143330	37691.348756
2	4953.146718	1610.845240	7032.128246	41704.155273
3	6936.393782	2189.984087	9560.352894	58115.109371
4	9637.954955	1941.023839	8473.519505	75939.204189
5	4676.663212	1766.677410	7712.411970	40449.054457

Figure 6: Sample Reorder Point Calculation for Selected Stores

Where:

- Lead Time = 7 days
- Z = 1.65 (for 95% service level)
- Demand Standard Deviation represents variability in historical sales

Average daily demand and demand variability were calculated store-wise. Stores with higher demand volatility required higher safety stock levels. For example, high-performing stores with large sales fluctuations resulted in higher reorder points compared to smaller stores.

This approach ensures sufficient inventory during replenishment lead time while reducing the risk of overstocking.

12. MODEL DEPLOYMENT USING STREAMLIT:

The final selected Random Forest model was serialized using Pickle and deployed locally using Streamlit to create an interactive sales prediction interface. The application allows users to input store-level features such as promotional status, competition details, and seasonal attributes to generate real-time daily sales forecasts.

Based on the predicted sales, the system also calculates a suggested reorder point using safety stock logic. This enables practical implementation of the forecasting model for inventory planning decisions.

Figure 7: Streamlit Application – Feature Input Interface.

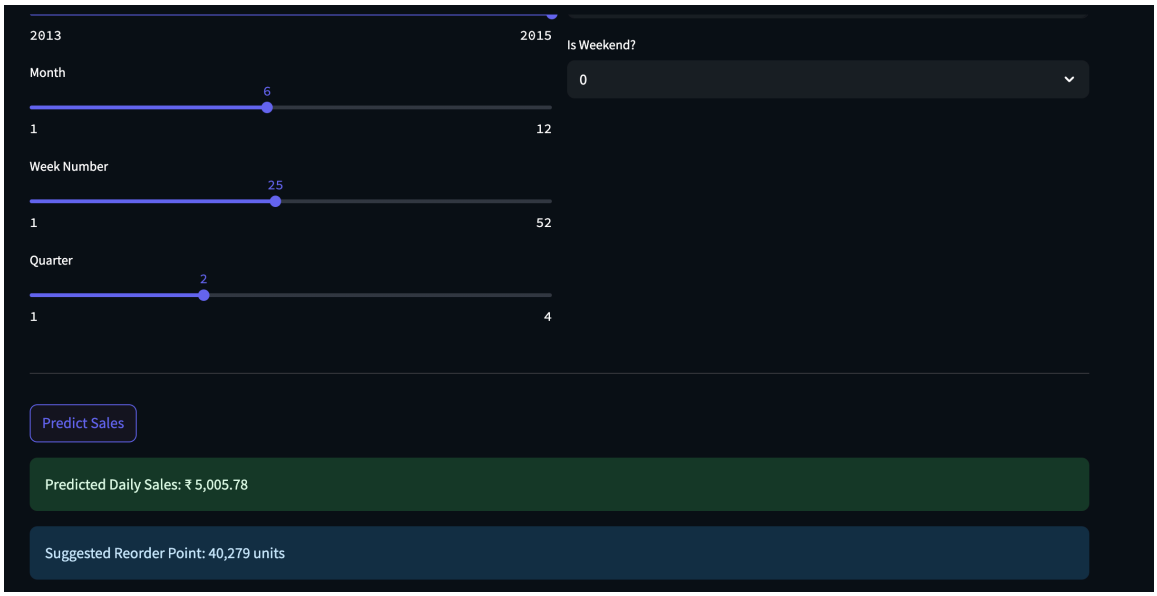

Figure 8: Streamlit Application – Prediction Output & Inventory Recommendation.

The deployment demonstrates the transition from analytical modeling to a usable business application.

13. BUSINESS IMPACT & CONCLUSION:

This project developed an end-to-end retail demand forecasting and inventory optimization system using SQL and Machine Learning. By combining historical sales analysis with predictive modeling, the solution improves demand visibility and supports data-driven inventory decisions.

Key outcomes of the project include:

- Identification of seasonal and promotional sales patterns using SQL
- Development and comparison of multiple regression models
- Selection of Random Forest as the best-performing model ($R^2 \approx 0.90$)
- Implementation of a safety stock and reorder point framework

From a business perspective, the system helps reduce stock-out risk, optimize inventory levels, and improve operational planning. The integration of analytics and machine learning demonstrates a practical, real-world retail forecasting solution.